

GIT AND GITHUB PART-2

1.Install Git

Step 1: Download Git

1. Open your web browser.
2. Go to the official Git website:
<https://git-scm.com/downloads>
3. Click **Download for Windows**.

Step 2: Run the Installer

1. Double-click the downloaded .exe file.
2. Click **Yes** if prompted

Step 4: Open Git Bash

After installation:

- Press **Windows**
- Search for **Git Bash**
- Open **Git Bash**

Verify git installation: `$ git --version`

```
brajk@RajKumari MINGW64 ~/OneDrive/Desktop
$ git --version
git version 2.37.1.windows.1
brajk@RajKumari MINGW64 ~/OneDrive/Desktop
$ |
```

2.Create a repo in github with README.md and .ignore file.

Objective: To create a new GitHub repository initialized with a README.md file and a .gitignore file for managing project documentation and ignoring unnecessary files.

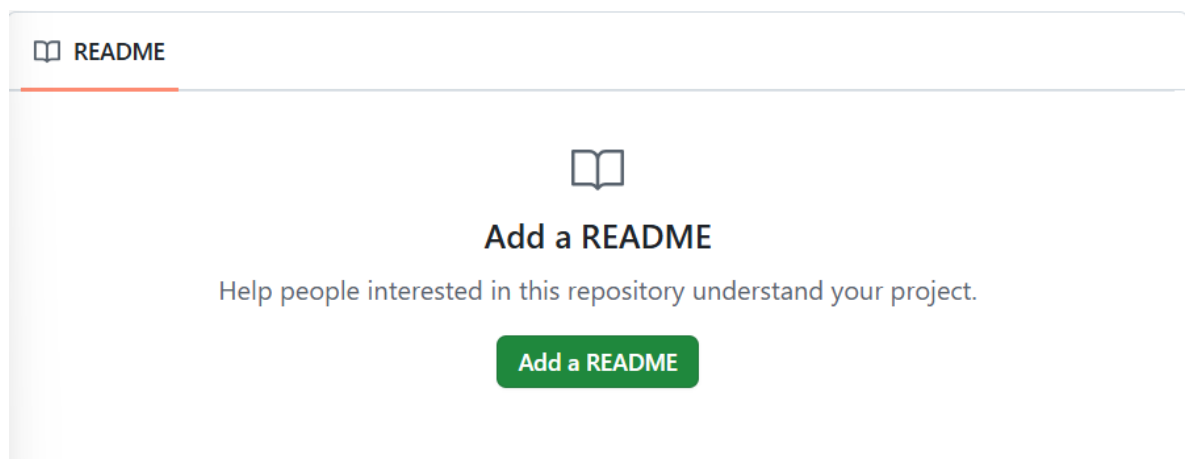
Purpose: The purpose of creating a GitHub repository with a README.md and .gitignore file is to organize the project, provide project information, and prevent unwanted files from being tracked by Git.

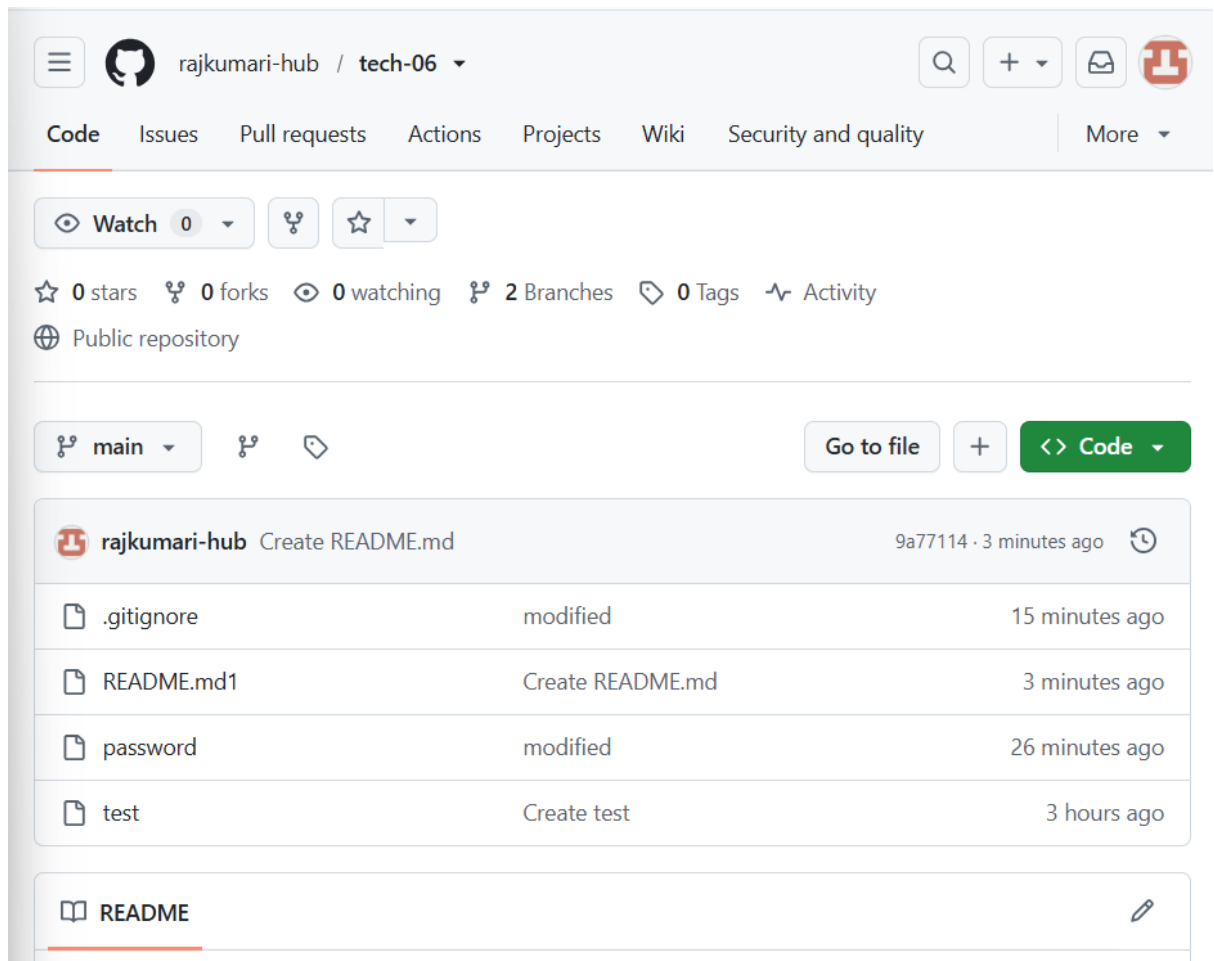
Step 1: Sign in to GitHub

1. Open your web browser.
2. Go to <https://github.com>
3. Sign in with your GitHub account.

Step 2: Create a New Repository

1. Click the + icon in the top-right corner.
2. Select New repository.





```
brajk@RajKumari MINGW64 ~/OneDrive/Desktop
$ cd tech-06

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ ls
password  test

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ cat .gitignore
password
secret
*.txt

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ |
```

3.Clone the created repo to local.

Objective: To clone an existing GitHub repository from the remote server to the local machine using Git.

Purpose: The purpose of cloning a repository is to create a complete local copy of the GitHub repository, including all

files, folders, branches, and commit history, so you can work on the project locally.

Step 1: Navigate to the Destination Folder

Move to the folder where you want to store the repository

Step 2: Clone the Repository

Run the following command:

```
$ git clone git@github.com:rajkumari-hub/tech-06.git
```

If we want to clone with separate branch, run the following command

```
$ git clone -b main git@github.com:rajkumari-hub/tech-06.git
```

```
brajk@RajKumari MINGW64 ~/OneDrive/Desktop
$ git clone -b main git@github.com:rajkumari-hub/tech-06.git
Cloning into 'tech-06'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (6/6), done.

brajk@RajKumari MINGW64 ~/OneDrive/Desktop
$ cd tech-06

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ ls
test

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git branch --list
* main

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ |
```

4. Create two files in local repo

Using touch command we can create two files

```
$ touch developer.txt engineer.txt
```

```
brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ touch developer.txt engineer.txt

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ ls
README.md1 developer.txt devops engineer.txt password test

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ |
```

5.Commit two files and push to central Repository.

Step 1: After creating two files, Check the status for untracking files

\$ git status

Step 2: Add the files to tracking files using command

\$ git add .

Step 3: Commit the files

\$ git commit -m "Added files"

Step 4: push the files into github repository

\$ git push -u origin main

```
brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ ls
README.md1 developer devops engineer password test

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        developer
        engineer

nothing added to commit but untracked files present (use "git add" to track)

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git add .

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git commit -m "added files"
[main 7d511d1] added files
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 developer
 create mode 100644 engineer

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ |
```

```

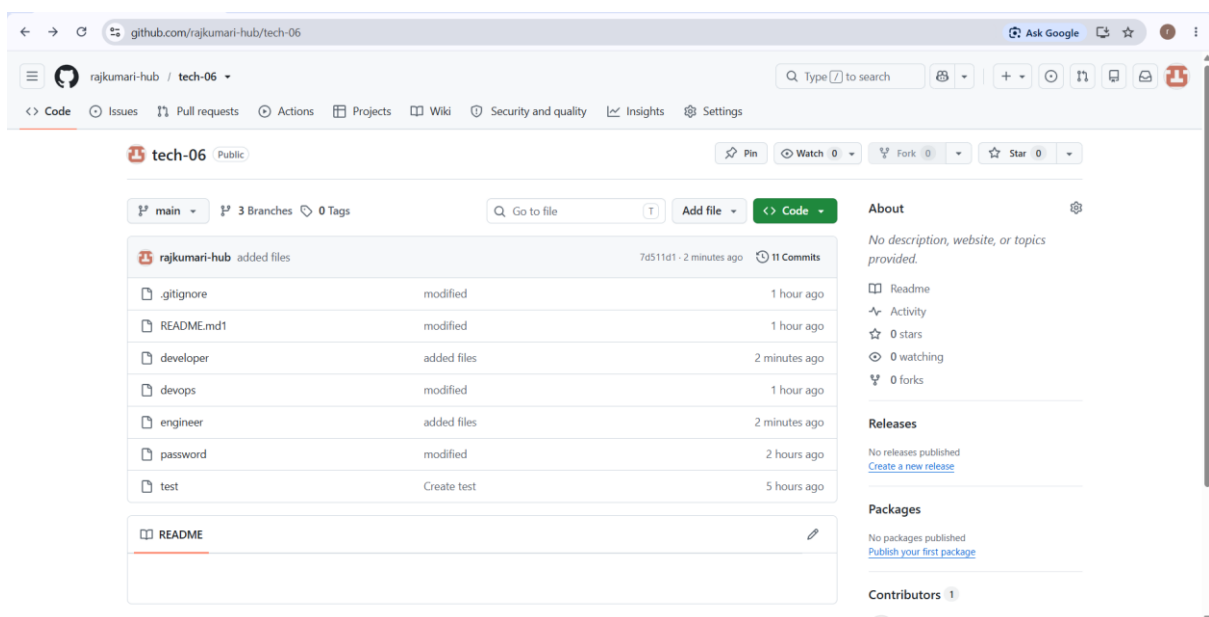
brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 271 bytes | 271.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:rajkumari-hub/tech-06.git
   cc41fa6..7d511d1  main -> main
branch 'main' set up to track 'origin/main'.

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ |

```



6. Create a branch in local and create a sample file and push to central.

Step1: create a new branch

\$ git branch feature

\$ git branch -list

Step2: Switch current branch new branch

\$ git checkout feature

Expected o/p: Switched to branch 'feature'

Step3: Create a sample file

\$ touch sample

```
brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git branch feature

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git branch --list
feature
* main

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git checkout feature
Switched to branch 'feature'

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$ ls
README.md1  developer  devops    engineer  password  test

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$ touch sample

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$ |
```

Step4: Check the Untracking files

\$ git status

\$ git add .

```
brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$ ls
README.md1  developer  devops    engineer  password  sample  test

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$ git status
On branch feature
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    sample

nothing added to commit but untracked files present (use "git add" to track)

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$ git add .
```

Step5: Commit the file

\$ git commit -m "add sample file"

```
brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$ git status
On branch feature
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   sample

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$ git commit -m "add sample file"
[feature 38ae343] add sample file
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 sample
```

Step6: push into github repo

\$ git push -u origin feature

```
brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$ git push -u origin feature
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 235 bytes | 235.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'feature' on GitHub by visiting:
remote:   https://github.com/rajkumari-hub/tech-06/pull/new/feature
remote:
To github.com:rajkumari-hub/tech-06.git
 * [new branch]      feature -> feature
branch 'feature' set up to track 'origin/feature'.
```

The screenshot shows the GitHub repository page for 'tech-06' by 'rajkumari-hub'. The repository is public and has 3 branches and 0 tags. A yellow banner indicates that the 'feature' branch has recent pushes 3 seconds ago, with a 'Compare & pull request' button. The file list shows '.gitignore' and 'README.md1' as modified files 1 hour ago. The 'About' section on the right states 'No description, website, or topics provided.' and shows 0 stars, 0 watching, and 0 forks.

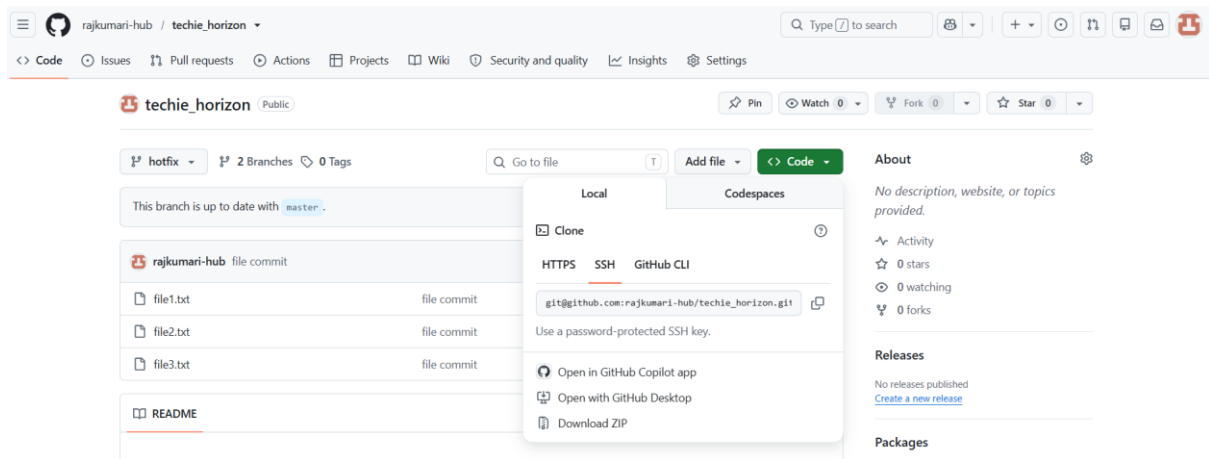
The screenshot shows the 'Pull requests' page for the 'tech-06' repository. A notification banner at the top says 'Label issues and pull requests for new contributors' and 'Now, GitHub will help potential first-time contributors discover issues labeled with good first issue'. Below this, there are filters for 'is:pr is:open' and buttons for 'Labels 9', 'Milestones 0', and 'New pull request'. The pull request list shows one open pull request titled 'add sample file' by 'rajkumari-hub', opened 1 minute ago.

7. Create a branch in github and clone that to local.

Step 1: Create a New Branch

1. Click the **Branch** drop-down in github(it usually shows main).
2. Type the new branch name.

Ex. hotfix



Step2: Clone into gitbash

\$ git clone [git@github.com:rajkumari-hub/techie_horizon.git](https://github.com/rajkumari-hub/techie_horizon.git)

```

brajk@RajKumari MINGW64 ~/OneDrive/Desktop
$ git clone git@github.com:rajkumari-hub/techie_horizon.git
Cloning into 'techie_horizon'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

brajk@RajKumari MINGW64 ~/OneDrive/Desktop
$ ls
18kq1a0102/      'Microsoft Edge.lnk'*      hello.py          techie_horizon/
DevOps/          UbuntuShare/               java_tutorial.pdf
'Docker Desktop.lnk'*  'Visual Studio Code.lnk'*  'rajkumari - Chrome.lnk'*
JavaTheCompleteReference.pdf  desktop.ini          tech-06/

brajk@RajKumari MINGW64 ~/OneDrive/Desktop
$ cd techie_horizon

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/techie_horizon (master)
$ git branch
* master

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/techie_horizon (master)
$ git fetch --all

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/techie_horizon (master)
$ git branch -a
* master
remotes/origin/HEAD -> origin/master
remotes/origin/hotfix
remotes/origin/master

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/techie_horizon (master)
$ git checkout hotfix
Switched to a new branch 'hotfix'
branch 'hotfix' set up to track 'origin/hotfix'.

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/techie_horizon (hotfix)
$ git branch
* hotfix
master

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/techie_horizon (hotfix)
$ ls
file1.txt file2.txt file3.txt

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/techie_horizon (hotfix)
$ |

```

8. Merge the created branch with master in git local.

```

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/hiring-app (hotfix)
$ git merge release
Already up to date.

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/hiring-app (hotfix)
$ ls
a b c d docker e f hello release z

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/hiring-app (hotfix)
$ |

```

```

added z

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/hiring-app (hotfix)
$ git checkout master
Switched to branch 'master'

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/hiring-app (master)
$ ls
docker hello master

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/hiring-app (master)
$ git cherry-pick fedb05e4
[master e046fffb] added in hotfix
Date: Wed Jul 8 12:28:26 2026 +0530
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 hotfix

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/hiring-app (master)
$ ls
docker hello hotfix master

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/hiring-app (master)
$

```

9. Merge the created branch with main in github by sending a pull request.

```

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$ git branch
* feature
  main

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$ git status
On branch feature
Your branch is up to date with 'origin/feature'.

nothing to commit, working tree clean

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$ git push -u origin feature
Everything up-to-date
branch 'feature' set up to track 'origin/feature'.

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git push -u origin feature
Everything up-to-date
branch 'feature' set up to track 'origin/feature'.

```

The screenshot shows the GitHub web interface for a repository named 'tech-06' owned by 'rajkumari-hub'. The repository is public and has 3 branches (main, feature, and tags) and 0 tags. A yellow banner at the top indicates that the 'feature' branch has recent pushes 3 seconds ago. A green button labeled 'Compare & pull request' is visible. Below the banner, the file list shows '.gitignore' and 'README.md1' as modified files, and 'docker' as a new file added 17 minutes ago. The right sidebar shows the repository's activity, including 11 commits, 0 stars, 0 watching, and 0 forks.

10. Create a file in local and send that to branch in github.

```
brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$ git status
On branch feature
Your branch is up to date with 'origin/feature'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
       hotel

nothing added to commit but untracked files present (use "git add" to track)

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$
```

```
brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$ git add .

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$ git commit -m "add hotel"
[feature c3e6b32] add hotel
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 hotel

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$ git status
On branch feature
Your branch is ahead of 'origin/feature' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$ git push -u origin feature
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 230 bytes | 230.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:rajkumari-hub/tech-06.git
   38ae343..c3e6b32  feature -> feature
branch 'feature' set up to track 'origin/feature'.

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$
```

The screenshot shows the GitHub interface for a repository named 'tech-06' owned by 'rajkumari-hub'. The repository is public and has 4 branches and 0 tags. The current branch is 'feature', which is 2 commits ahead of 'main'. The commit history shows a recent commit 'add hotel' by 'rajkumari-hub' at 'c3e6b32' 1 minute ago. The file list includes .gitignore, README.md1, developer, devops, engineer, hotel, password, sample, and test. The 'About' section on the right indicates no description, website, or topics are provided. The 'Releases' and 'Packages' sections also show no published content. The 'Contributors' section lists 'rajkumari-hub' as the sole contributor.

File	Change	Time
.gitignore	modified	2 hours ago
README.md1	modified	2 hours ago
developer	added files	1 hour ago
devops	modified	2 hours ago
engineer	added files	1 hour ago
hotel	add hotel	1 minute ago
password	modified	3 hours ago
sample	add sample file	1 hour ago
test	Create test	6 hours ago

11. Clone only a branch from github to local.

Objective: To clone a specific branch (feature) from a remote GitHub repository to the local machine using the `git clone -b` command, so that work can begin directly on that branch.

Purpose: The purpose of cloning a specific branch is to download only the desired branch from the remote repository and set it as the active branch in the local repository. This allows developers to work on a particular feature or task without switching branches after cloning.

\$ `git clone -b feature git@github.com:rajkumari-hub/tech-06.git`

```
brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$ git clone -b feature git@github.com:rajkumari-hub/tech-06.git
Cloning into 'tech-06'...
remote: Enumerating objects: 34, done.
remote: Counting objects: 100% (34/34), done.
remote: Compressing objects: 100% (23/23), done.
remote: Total 34 (delta 6), reused 22 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (34/34), 5.71 KiB | 2.85 MiB/s, done.
Resolving deltas: 100% (6/6), done.

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$ ls
README.md1  developer  devops    engineer  hotel     password  sample    tech-06/  test

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (feature)
$ s|
```

12. Create a file with all passwords and make that untrackable with git.

Objective: To create a file containing passwords and configure Git to ignore the file so that it is not tracked or pushed to the GitHub repository.

Purpose: The purpose of this task is to protect sensitive information (such as passwords, API keys, or secrets) by preventing Git from tracking or uploading the file to the remote repository.

```
brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git commit -m "added password"
[main bef63c1] added password
1 file changed, 2 insertions(+)
create mode 100644 password

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 303 bytes | 303.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:rajkumari-hub/tech-06.git
a74ec0f..bef63c1 main -> main
branch 'main' set up to track 'origin/main'.

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
```

```
brajk@RajKumari MINGW64 ~/OneDrive/Desktop
$ cd tech-06

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ ls
password test

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ cat .gitignore
password
secret
*.txt

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ |
```

rajkumari-hub / tech-06

🔍

+

📧

🔗

Code

Issues

Pull requests

Actions

Projects

Wiki

Security and quality

More

👁 Watch 0

🔗

★

☆ 0 stars 🔗 0 forks 👁 0 watching 🌿 2 Branches 🏷 0 Tags 🔄 Activity

🌐 Public repository

🌿 main

🔗

📁

Go to file

+

<> Code

🔗 rajkumari-hub Create README.md

9a77114 · 3 minutes ago

📄 .gitignore

modified

15 minutes ago

📄 README.md1

Create README.md

3 minutes ago

📄 password

modified

26 minutes ago

📄 test

Create test

3 hours ago

📖 README

✎

rajkumari-hub / tech-06

🔍

+

📧

🔗

Code

Issues

Pull requests

Actions

Projects

Wiki

Security and quality

More

← Files

🌿 main

tech-06 / password

...

🔗 rajkumari-hub added password

bef63c1 · 2 minutes ago

2 lines (2 loc) · 33 Bytes

Code

Blame

Raw

📄

📄

✎

🔗

1 username:rajkumari

2 password:raji

13. Make a commit and make that commit reset without saving changes

Objective: To create a commit and then remove the commit along with all its changes, restoring the repository to the state before the commit.

Purpose: The purpose of `git reset --hard` is to permanently discard the most recent commit and all the changes associated with it. This is useful when a commit was made by mistake and you no longer need the changes.


```

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06/tech-06 (feature)
$ ls
README.md1 developer devops engineer hotel password sample test

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06/tech-06 (feature)
$ touch restfile

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06/tech-06 (feature)
$ ls
README.md1 developer devops engineer hotel password restfile sample test

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06/tech-06 (feature)
$ git status
On branch feature
Your branch is up to date with 'origin/feature'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    restfile

nothing added to commit but untracked files present (use "git add" to track)

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06/tech-06 (feature)
$ git add .

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06/tech-06 (feature)
$ git commit -m "add reset file"
[feature bed51da] add reset file
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 restfile

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06/tech-06 (feature)
$ git log --oneline
bed51da (HEAD -> feature) add reset file
c3e6b32 (origin/feature) add hotel
38ae343 add sample file
7d511d1 (origin/main, origin/HEAD) added files
cc41fa6 Merge branch 'main' of github.com:rajkumari-hub/tech-06
7bdc522 modified
0a45bd9 (origin/test) Create devops
9a77114 Create README.md
2e71ac7 modified
737c10a modified
a7a4411 modified
88489db added .gitignore
bef63c1 added password
a74ec0f Create test

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06/tech-06 (feature)
$ |

```

Here, Reset the Last Commit Without Saving Changes by using command

```
$ git reset --hard HEAD~1
```

For checking

```
$ git log --oneline
```

Here, there no untracking file

```
$ git status
```

```

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06/tech-06 (feature)
$ git reset --hard HEAD~1
HEAD is now at c3e6b32 add hotel

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06/tech-06 (feature)
$ git log --oneline
c3e6b32 (HEAD -> feature, origin/feature) add hotel
38ae343 add sample file
7d511d1 (origin/main, origin/HEAD) added files
cc41fa6 Merge branch 'main' of github.com:rajkumari-hub/tech-06
7bdc522 modified
0a45bd9 (origin/test) Create devops
9a77114 Create README.md
2e71ac7 modified
737c10a modified
a7a4411 modified
88489db added .gitignore
bef63c1 added password
a74ec0f Create test

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06/tech-06 (feature)
$ git status
On branch feature
Your branch is up to date with 'origin/feature'.

nothing to commit, working tree clean

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06/tech-06 (feature)
$ |

```

14. Revert a committed commit to the older version

```

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06/tech-06 (feature)
$ git revert 38ae343
[feature dc8db46] Revert "add sample file"
1 file changed, 0 insertions(+), 0 deletions(-)
delete mode 100644 sample

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06/tech-06 (feature)
$ git log --oneline
dc8db46 (HEAD -> feature) Revert "add sample file"
c3e6b32 (origin/feature) add hotel
38ae343 add sample file
7d511d1 (origin/main, origin/HEAD) added files
cc41fa6 Merge branch 'main' of github.com:rajkumari-hub/tech-06
7bdc522 modified
0a45bd9 (origin/test) Create devops
9a77114 Create README.md
2e71ac7 modified
737c10a modified
a7a4411 modified
88489db added .gitignore
bef63c1 added password
a74ec0f Create test

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06/tech-06 (feature)
$ |

```

Revert "add sample file"

This reverts commit 38ae343ffe5f19ffa64c332c1eee23c32e23e132.

Please enter the commit message for your changes. Lines starting
with '#' will be ignored, and an empty message aborts the commit.

#

On branch feature
Your branch is up to date with 'origin/feature'.

#

Changes to be committed:
deleted: sample

#

~

~

```
brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06/tech-06 (feature)
$ git status
On branch feature
Your branch is ahead of 'origin/feature' by 1 commit.
(use "git push" to publish your local commits)

nothing to commit, working tree clean

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06/tech-06 (feature)
$
```

15. Push a file to stash without saving the changes and work on another file.

Objective: To temporarily save uncommitted changes using Git Stash, switch to another task, and continue working on a different file without committing the current changes.

Purpose: The purpose of Git Stash is to temporarily store uncommitted changes in a stack, allowing you to work on another file or branch without losing your current work. You can restore the stashed changes later whenever needed.

```
brajk@RajKumari MINGW64 ~/OneDrive/Desktop/hiring-app (master)
$ ls
alphabet  docker  hello  hotfix  master

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/hiring-app (master)
$ git stash
Saved working directory and index state WIP on master: 2ca0cc9 changed the commit message by using amend cmd

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/hiring-app (master)
$ ls
docker  hello  hotfix  master

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/hiring-app (master)
$
```

```
brajk@RajKumari MINGW64 ~/OneDrive/Desktop/hiring-app (master)
$ git stash list
stash@{0}: WIP on master: 2ca0cc9 changed the commit msg by using amend cmd

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/hiring-app (master)
$ git stash pop
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   alphabet

Dropped refs/stash@{0} (546c8381370ea38abb40ca0ec97fc8d9b173d910)

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/hiring-app (master)
$ ls
alphabet  docker  hello  hotfix  master

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/hiring-app (master)
$
```

16. Undo the stash file and start working on that again.

Objective: To restore the previously stashed changes back to the working directory and continue working on the same file.

Purpose: The purpose of undoing a stash is to retrieve temporarily saved changes and resume work without losing any modifications. Git provides git stash apply and git stash pop to restore stashed changes.

```

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ ls
README.md1 developer devops engineer password resetfile tech-06/ test

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ echo "new content" >> developer

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   developer

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        resetfile
        tech-06/

no changes added to commit (use "git add" and/or "git commit -a")

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git stash
warning: in the working copy of 'developer', LF will be replaced by CRLF the next time Git touches it
Saved working directory and index state WIP on main: 7d511d1 added files

```

```

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git stash list
stash@{0}: WIP on main: 7d511d1 added files

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git stash pop
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   developer

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        resetfile
        tech-06/

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (ea647b5a582602c5837ac72ba01c4ce2b7aa0056)

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

```

17. Generate a ssh-keygen and configure into github.

Objective: To generate an SSH key pair on the local machine and configure the public key in GitHub to enable secure authentication without entering a username and password for every Git operation.

Purpose: The purpose of using SSH keys is to establish a secure connection between the local machine and GitHub. Once configured, you can clone, push, and pull repositories using SSH authentication.

Step 1: Generate a New SSH Key

```
$ ssh-keygen
```

Check, if already exist

```
$ ls ~/.ssh
```

```
$ cat ~/.ssh/id_rsa.pub
```

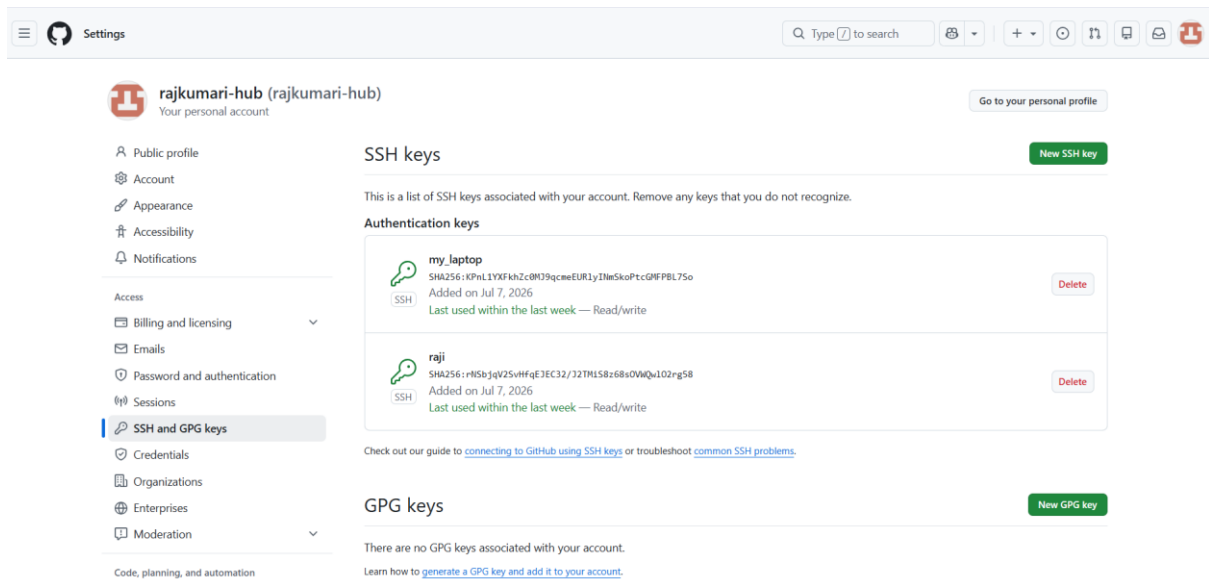
```
brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ ls ~/.ssh
id_rsa  id_rsa.pub  known_hosts  known_hosts.old

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQACj5jsCdVZjKNVWgZLN6YeifGkWpP4NWq+4DWyWRmBj3/gFoU91hNr+nuWshG1pj/w
DGVNTUMCGjPTRfQqnsvhez6r9PQsvADtws6WHN1ZPRYyCuSzVUizYrLB14UXIC0ZYv9zN8PwXZ5iq5kDZX2ctkEhiZ3PgnSiChk2g
Q4u5G1S9WsKw09A6T0kyr438Cx7r+/GRpnBy6c3dj/GA0V1Jegng6GtTFc6pMkqy5jij0e7ZF89BYMvmky5sfoUG6Z5Erw33TJHiOu
vze3qCeT6SgfZaGwrhCxKijSj55WAcKig8kbqna1Sig/grG9Yo4fQJGI+UNE019bQNN08xdCfKi2sLLsqMxnjmuRYA798JgV28xhMiq
c5L2Hu8WwI2A1yb42L003QeQ3kXsIqL5JQxchqALJp8xQxcI5DiARm8C6VxpaKL1pwhd2uC8DmNowVoBUMloOasj9zYPxd3EDma6Lk1
z16/aqZVWN8151MjiAub9mU21po1U7xjHLhtpZADHBStJY5pU/Xn3tAV1XA9HpR7cFAAHoFyiIW71tR5gpR7Lhjg2g9NCfPKCwt5hKo
N/4dJ8Jy3cgU29XMwUna2XUMUofDhY4KyfT4Mpd96DDdpYTYA4owzdRIK7M7BRK4/3ePXY+RBnSjL51b9+7yK02DnNTZUsium4Tf9Xp
w== brajk@RajKumari

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ |
```

Step2: Add the SSH Key to GitHub

1. Log in to **GitHub**.
2. Click your **Profile** icon.
3. Select **Settings**.
4. Click **SSH and GPG keys**.
5. Click **New SSH key**.
6. Enter a title, for example



\$ git remote add origin [git@github.com:rajkumari-hub/tech-06.git](https://github.com/rajkumari-hub/tech-06.git)

\$ git remote -v

```
brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git remote -v
origin  git@github.com:rajkumari-hub/tech-06.git (fetch)
origin  git@github.com:rajkumari-hub/tech-06.git (push)

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ |
```

18. Configure webhooks to github.

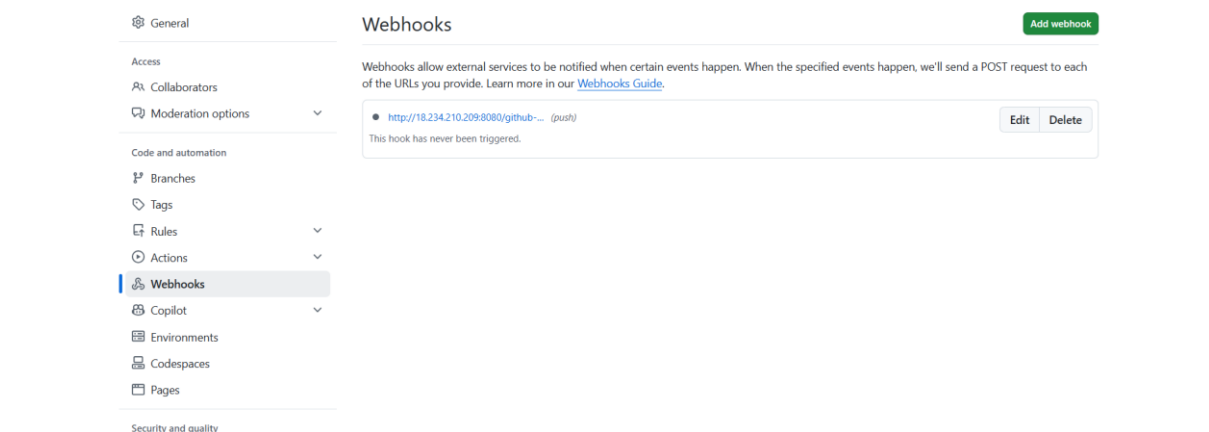
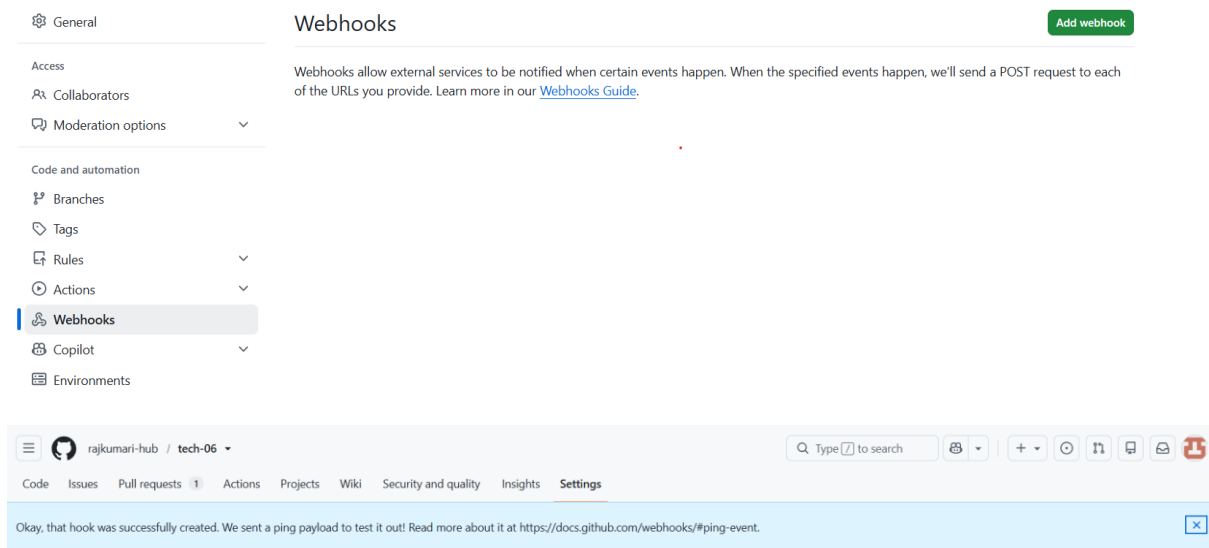
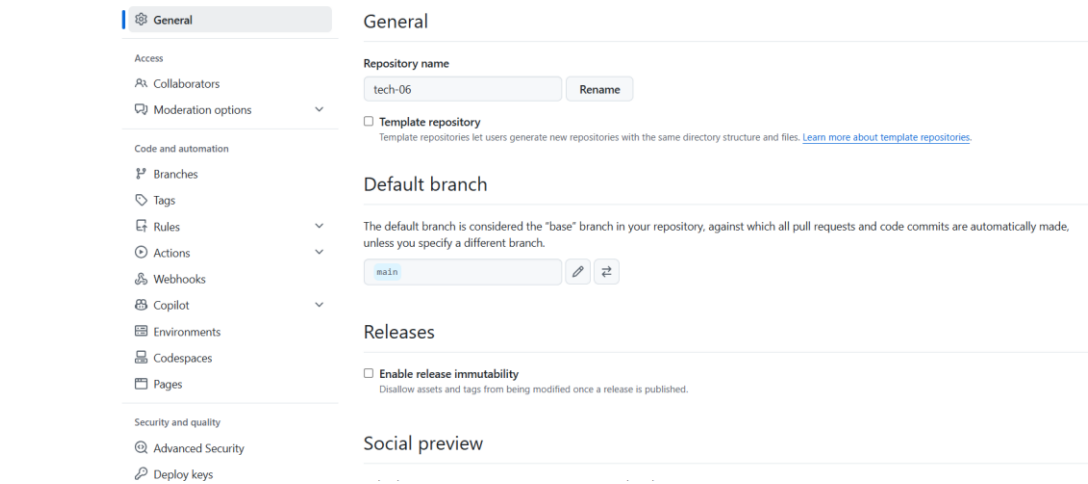
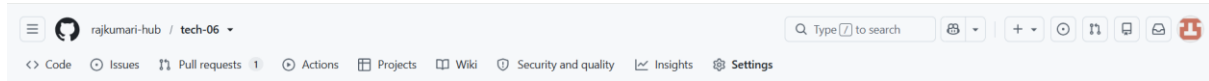
Objective: To configure a GitHub Webhook so that GitHub automatically sends event notifications (such as push, pull request, or release events) to an external application whenever specific actions occur in a repository.

Purpose: GitHub Webhooks automate workflows by notifying external services whenever an event occurs in a GitHub repository. They are commonly used for Continuous Integration (CI), Continuous Deployment (CD), Jenkins, Slack notifications, deployment pipelines, and custom applications.

Steps

1. Sign in to your GitHub account.
2. Open the repository where you want to configure the webhook.
3. Click **Settings**.
4. In the left sidebar, click **Webhooks**.
5. Click **Add webhook**.
6. In the **Payload URL** field, enter the URL of your application (the endpoint that will receive webhook events).
7. Select **Content type** as **application/json**.
8. (Optional) Enter a **Secret** to secure the webhook.
9. Choose the events that should trigger the webhook:
 - **Just the push event** (recommended), or
 - **Send me everything**, or
 - **Let me select individual events**.
10. Ensure the **Active** checkbox is selected.
11. Click **Add webhook** to save the configuration.
12. Push a commit or perform the selected event to test the webhook.
13. Verify the webhook delivery by going to **Settings** → **Webhooks** → **Select the webhook** → **Recent Deliveries**.

A **200 OK** response indicates the webhook is working successfully.



19. Basic understanding of .git file

Objective: To understand the purpose of the .git folder and how Git uses it to manage version control for a repository.

Purpose: The .git folder is the **hidden directory** that stores all the information Git needs to track changes, manage branches, store commits, and maintain the complete history of a repository. Without the .git folder, a directory is just a normal folder and is no longer a Git repository

What is the .git Folder?

- The .git folder is created automatically when you run:

\$ git init

- It is located in the root directory of your Git repository.
- It is a hidden folder (because its name starts with a .)

```

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git init
Reinitialized existing Git repository in C:/Users/brajk/OneDrive/Desktop/tech-06/.git/

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ ls -la
total 22
drwxr-xr-x 1 brajk 197609  0 Jul  8 20:21 ./
drwxr-xr-x 1 brajk 197609  0 Jul  8 19:30 ../
drwxr-xr-x 1 brajk 197609  0 Jul  8 20:48 .git/
-rw-r--r-- 1 brajk 197609 25 Jul  8 15:55 .gitignore
-rw-r--r-- 1 brajk 197609  2 Jul  8 15:55 README.md1
-rw-r--r-- 1 brajk 197609 13 Jul  8 20:21 developer
-rw-r--r-- 1 brajk 197609  2 Jul  8 15:55 devops
-rw-r--r-- 1 brajk 197609  0 Jul  8 17:19 engineer
-rw-r--r-- 1 brajk 197609 37 Jul  8 15:15 password
-rw-r--r-- 1 brajk 197609  0 Jul  8 19:42 resetfile
drwxr-xr-x 1 brajk 197609  0 Jul  8 20:16 tech-06/
-rw-r--r-- 1 brajk 197609  7 Jul  8 15:15 test

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ ls -la .git
total 36
drwxr-xr-x 1 brajk 197609  0 Jul  8 20:48 ./
drwxr-xr-x 1 brajk 197609  0 Jul  8 20:21 ../
-rw-r--r-- 1 brajk 197609 41 Jul  8 20:21 AUTO_MERGE
-rw-r--r-- 1 brajk 197609 10 Jul  8 18:44 COMMIT_EDITMSG
-rw-r--r-- 1 brajk 197609 92 Jul  8 17:05 FETCH_HEAD
-rw-r--r-- 1 brajk 197609 21 Jul  8 20:17 HEAD
-rw-r--r-- 1 brajk 197609 41 Jul  8 20:20 ORIG_HEAD
-rw-r--r-- 1 brajk 197609 41 Jul  8 15:55 REBASE_HEAD
-rw-r--r-- 1 brajk 197609 363 Jul  8 20:48 config
-rw-r--r-- 1 brajk 197609 73 Jul  8 15:15 description
drwxr-xr-x 1 brajk 197609  0 Jul  8 15:15 hooks/
-rw-r--r-- 1 brajk 197609 566 Jul  8 20:21 index
drwxr-xr-x 1 brajk 197609  0 Jul  8 15:15 info/
drwxr-xr-x 1 brajk 197609  0 Jul  8 15:15 logs/
drwxr-xr-x 1 brajk 197609  0 Jul  8 20:20 objects/
-rw-r--r-- 1 brajk 197609 180 Jul  8 15:15 packed-refs
drwxr-xr-x 1 brajk 197609  0 Jul  8 20:21 refs/

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ |

```

20. Check all the logs of git.

Objective: To view the commit history, branch movements, reference updates, and other Git activity using Git log commands.

Purpose: The purpose of Git logs is to track all changes made in a repository, including commits, branch operations, merges, authors, dates, and reference updates. Git logs help developers understand the project's history and troubleshoot issues.

```

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git log
commit 7d511d127025952130be648d22180cc5274ff4c4 (HEAD -> main, origin/main, origin/HEAD)
Author: rajkumari <brajkumari7675@gmail.com>
Date: Wed Jul 8 17:20:08 2026 +0530

    added files

commit cc41fa63e0e1d7283dd3321a46df3adb4c1cd7dd
Merge: 7bdc522 0a45bd9
Author: rajkumari <brajkumari7675@gmail.com>
Date: Wed Jul 8 17:02:21 2026 +0530

    Merge branch 'main' of github.com:rajkumari-hub/tech-06

commit 7bdc522542849793fe9209a15028aba95ac505d2
Author: rajkumari <brajkumari7675@gmail.com>
Date: Wed Jul 8 15:02:47 2026 +0530

    modified

    modified

    Create README.md

    Create devops

commit 0a45bd97c08e320f534c29bf8caef4f1d8dc0547 (origin/test)
Author: rajkumari-hub <brajkumari7675@gmail.com>
Date: Wed Jul 8 15:28:44 2026 +0530

    Create devops

commit 9a77114734215ceecef11417539d2ec9f27b293c
Author: rajkumari-hub <brajkumari7675@gmail.com>
Date: Wed Jul 8 15:20:58 2026 +0530

    Create README.md

commit 2e71ac740b4609e95761de64a7810dfceb81c67a
Author: rajkumari <brajkumari7675@gmail.com>
Date: Wed Jul 8 15:08:07 2026 +0530

    modified

commit 737c10acc3fd350d2e4d496afa050e84dd5160f6
Author: rajkumari <brajkumari7675@gmail.com>
Date: Wed Jul 8 15:02:47 2026 +0530

    modified

commit a7a44110bd1a4bc52e4f54f67413886f5ed1c026
Author: rajkumari <brajkumari7675@gmail.com>
Date: Wed Jul 8 14:57:57 2026 +0530

    modified

```

21. Rename the commit message.

```

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git log -4
commit 7d511d127025952130be648d22180cc5274ff4c4 (HEAD -> main, origin/main, origin/HEAD)
Author: rajkumari <brajkumari7675@gmail.com>
Date: Wed Jul 8 17:20:08 2026 +0530

    added files

commit cc41fa63e0e1d7283dd3321a46df3adb4c1cd7dd
Merge: 7bdc522 0a45bd9
Author: rajkumari <brajkumari7675@gmail.com>
Date: Wed Jul 8 17:02:21 2026 +0530

    Merge branch 'main' of github.com:rajkumari-hub/tech-06

```

```

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git log --oneline
7d511d1 (HEAD -> main, origin/main, origin/HEAD) added files
cc41fa6 Merge branch 'main' of github.com:rajkumari-hub/tech-06
7bdc522 modified
0a45bd9 (origin/test) Create devops
9a77114 Create README.md
2e71ac7 modified
737c10a modified
a7a4411 modified
88489db added .gitignore
bef63c1 added password
a74ec0f Create test

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git commit --amend -m "Added developer and engineer files"
[main 5793655] Added developer and engineer files
Date: Wed Jul 8 17:20:08 2026 +0530
2 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 developer
create mode 100644 engineer

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git log --oneline
5793655 (HEAD -> main) Added developer and engineer files
cc41fa6 Merge branch 'main' of github.com:rajkumari-hub/tech-06
7bdc522 modified
0a45bd9 (origin/test) Create devops
9a77114 Create README.md
2e71ac7 modified
737c10a modified
a7a4411 modified
88489db added .gitignore
bef63c1 added password
a74ec0f Create test

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ |

```

22. Merge multiple commits into single commit.

Objective: Combine multiple commits into one clean commit to keep the Git history simple and readable.

Purpose: This is useful when you have made several small commits while developing a feature and want to merge them into a single meaningful commit before pushing or creating a pull request.

\$ git log -oneline

\$ git rebase -i HEAD~4

```

pick 737c10a modified
squash 2e71ac7 modified
squash 9a77114 Create README.md
squash 0a45bd9 Create devops

# Rebase a7a4411..0a45bd9 onto a7a4411 (4 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                               commit's log message, unless -C is used, in which case
#                               keep only this commit's message; -c is same as -C but
#                               opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name

```

```

# This is a combination of 4 commits.
# This is the 1st commit message:

modified

# This is the commit message #2:

modified

# This is the commit message #3:

Create README.md
# This is the commit message #4:

Create devops

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# Date:      Wed Jul 8 15:02:47 2026 +0530
#
# interactive rebase in progress; onto a7a4411

```

```

brajk@RajKumari MINGW64 ~/OneDrive/Desktop/tech-06 (main)
$ git rebase -i HEAD~4
[detached HEAD 7bdc522] modified
Date: Wed Jul 8 15:02:47 2026 +0530
3 files changed, 4 insertions(+)
create mode 100644 README.md1
create mode 100644 devops
Successfully rebased and updated refs/heads/main.

```

```

commit 7bdc522542849793fe9209a15028aba95ac505d2 (HEAD -> main)
Author: rajkumari <brajkumari7675@gmail.com>
Date:   Wed Jul 8 15:02:47 2026 +0530

```

```

modified
modified
Create README.md
Create devops

```

